Kubernetes on Your Local Machine

Run an Express app on a local cluster: every resource, every YAML field, ingress included

Big picture

```
Browser (http://localhost)
   |
   v
Ingress Controller (nginx)  <-- reads the Ingress rules
   |
   v
Service (ClusterIP, port 80)   <-- stable name + IP
   |
   v
Pods x2 (container port 3000)  <-- created and kept alive by the Deployment
```

You write three YAML files (Deployment, Service, Ingress), apply them with `kubectl`, and Kubernetes keeps the real state matching what the files describe.

Key terms

Every Kubernetes object has a purpose and a typical lifecycle. Learn this table first, because the rest of the guide uses these words constantly.

Resource	Purpose	Lifecycle
Cluster	The whole Kubernetes environment: control plane plus nodes	Usually shared
Node	A machine (real or virtual) that runs your workloads	Cluster-level
Namespace	Logical isolation inside a cluster (like folders)	Per environment/team
Deployment	Manages application replicas, rolling updates and self-healing	Per application
Pod	Smallest unit. Runs one or more containers	Automatically created
Service	Stable name and IP for reaching a changing set of Pods	Per application
ConfigMap	Non-secret configuration (env vars, config files)	As required
Secret	Sensitive configuration (passwords, tokens)	As required
Ingress	HTTP/HTTPS routing rules (host and path to Service)	Per application/routing setup
Ingress Controller	The program (e.g. nginx) that reads Ingress rules and does the routing	Usually shared
Persistent Volume	Storage that outlives Pods (claimed with a PersistentVolumeClaim)	As required
HPA	Horizontal Pod Autoscaler: adds or removes Pods based on load	Per application
ServiceAccount / RBAC	Identity for Pods and the permissions they are allowed	As required

Two things are easy to confuse:

- **Ingress vs Ingress Controller.** An Ingress is only a rule written in YAML. Nothing happens until an Ingress Controller is installed to enforce it.
- **Deployment vs Pod.** You almost never create Pods by hand. You create a Deployment and it creates, replaces and scales Pods for you.

Step 0: Prerequisites

- **Docker Desktop** (or Docker Engine on Linux)
- **kubectl**, the command-line client for Kubernetes
- **A local cluster tool**: Docker Desktop's built-in Kubernetes, minikube, or kind (see Step 1)
- **Helm** (only if you choose the Helm route for the ingress controller in Step 6)
- An Express app with a Dockerfile that listens on port 3000

Check the tools:

```
docker --version
kubectl version --client
```

Step 1: Pick and create a local cluster

You only need **one** of these. All three give you a real single-node Kubernetes cluster on your laptop.

Option	Best for	Create the cluster
Docker Desktop	Easiest. No extra tool to install	Settings > Kubernetes > **Enable Kubernetes** > Apply
minikube	Beginners. Has handy add-ons	`minikube start --driver=docker`
kind	Kubernetes in Docker. Fast, good for CI	`kind create cluster --name demo` (use the config in Step 6 if you want ingress)

Confirm kubectl is talking to your cluster:

```
kubectl config current-context     # docker-desktop, minikube, or kind-demo
kubectl get nodes                  # STATUS should be Ready
```

Step 2: Build the image and make the cluster able to see it

Your Deployment uses the image name `kuber-demo`, which exists only on your machine (not in a registry). So the cluster must be able to find it locally. Example Dockerfile:

Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "index.js"]
```

Build

```
docker build -t kuber-demo .
```

Cluster	Make the image available
Docker Desktop	Nothing to do. The cluster shares your Docker image store
minikube	`minikube image load kuber-demo`
kind	`kind load docker-image kuber-demo --name demo`

> **Rebuilt the image?** Load it into minikube or kind again, then run `kubectl rollout restart deployment kuber-demo-deployment` so the Pods pick it up.

Step 3: The Deployment (deployment.yaml)

A Deployment says *"I want 2 copies of this container running, always."* If a Pod crashes, the Deployment starts a new one. When you change the image, it replaces Pods gradually (rolling update).

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kuber-demo-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: express
  template:
    metadata:
      labels:
        app: express
    spec:
      containers:
      - name: kuber-demo-container
        image: kuber-demo
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 3000
        resources:
          requests:
            memory: "64Mi"
            cpu: "250m"
          limits:
            memory: "128Mi"
            cpu: "500m"
```

Every field explained

Field	What it means
`apiVersion: apps/v1`	Which API group and version defines this object. Deployments live in **apps/v1**.
`kind: Deployment`	The type of object you are creating.
`metadata.name`	The unique name of this Deployment inside its namespace. You use it in kubectl commands.
`spec.replicas`	How many identical Pods to keep running. Here: 2.
`spec.selector.matchLabels`	How the Deployment finds the Pods it owns. **Must match** the labels in the Pod template, or Kubernetes rejects the file.
`spec.template`	The blueprint used to create every Pod.
`template.metadata.labels`	Labels stuck on each Pod. The **Service** uses these same labels to find the Pods. Here: `app: express`.
`template.spec.containers`	The list of containers inside each Pod. Most Pods have just one.
`name`	The container's name inside the Pod (used in `kubectl logs -c`).
`image`	Which image to run. With no tag, Kubernetes assumes `:latest`.
`imagePullPolicy: IfNotPresent`	Use the local image if it exists, pull only if missing. Other values: `Always` (always pull) and `Never` (never pull). This is why local images work.
`ports.containerPort`	The port the app listens on inside the container (3000). It documents the port. It does **not** publish it. The Service's `targetPort` does the real routing.
`resources.requests`	The minimum the Pod is **guaranteed**. The scheduler uses it to pick a Node. Memory `64Mi` = 64 mebibytes. CPU `250m` = 250 millicores = 0.25 of a core. The HPA also measures CPU against this number.
`resources.limits`	The maximum the container may use. Going over the memory limit gets the container killed (`OOMKilled`). Going over the CPU limit only slows it down (throttling).

Step 4: The Service (service.yaml)

Pods come and go and their IPs change. A Service gives them one stable name and IP, and spreads traffic across all healthy Pods that match its selector.

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: kuber-demo-service
spec:
  selector:
    app: express
  type: ClusterIP
  ports:
  - name: http
    port: 80
    targetPort: 3000
```

Every field explained

Field	What it means
`apiVersion: v1`	Services belong to the core API group, so the version is just `v1`.
`kind: Service`	The type of object.
`metadata.name`	The Service name. It also becomes a DNS name: other Pods can call `http://kuber-demo-service` (full form `kuber-demo-service.default.svc.cluster.local`).
`spec.selector`	Send traffic to Pods that have this label. `app: express` matches the Pods from the Deployment. **If the labels do not match, the Service has no endpoints and nothing works.**
`spec.type: ClusterIP`	Reachable only **inside** the cluster (the default). Other types: `NodePort` (opens a port on every Node) and `LoadBalancer` (asks the cloud or Docker Desktop for an external IP). With an Ingress in front, ClusterIP is the right choice.
`ports.name`	A label for the port. Required when a Service has more than one port.
`ports.port`	The port the **Service** listens on (80).
`ports.targetPort`	The port on the **container** that traffic is forwarded to (3000). Must match what your app listens on.

Step 5: Install the Ingress Controller

An Ingress does nothing on its own. You must install a controller first. This guide uses the **NGINX Ingress Controller**, because the Ingress file uses `ingressClassName: nginx`.

> **Heads up:** the Kubernetes project retired **ingress-nginx** in March 2026. It gets no more bug fixes or security patches. It still works and is fine for learning on your own laptop, but do not use it for anything public or production. For real use, look at the Gateway API or another Ingress controller.

Option A: Docker Desktop (Helm)

```
helm upgrade --install ingress-nginx ingress-nginx \
  --repo https://kubernetes.github.io/ingress-nginx \
  --namespace ingress-nginx --create-namespace
```

Docker Desktop gives the controller's LoadBalancer Service the address `localhost`, so `http://localhost` works with no extra steps.

Option B: minikube

```
minikube addons enable ingress
minikube tunnel        # keep this running in a second terminal
```

On macOS and Windows with the Docker driver, `minikube tunnel` is what makes `http://localhost` reach the controller.

Option C: kind

kind runs the cluster inside a Docker container, so you must map ports 80 and 443 to your machine **when creating the cluster**. Save this as `kind-config.yaml`:

kind-config.yaml

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  kubeadmConfigPatches:
  - |
    kind: InitConfiguration
    nodeRegistration:
      kubeletExtraArgs:
        node-labels: "ingress-ready=true"
  extraPortMappings:
  - containerPort: 80
    hostPort: 80
    protocol: TCP
  - containerPort: 443
    hostPort: 443
    protocol: TCP
```

```
kind create cluster --name demo --config kind-config.yaml

INGRESS=https://raw.githubusercontent.com/kubernetes/ingress-nginx/main
kubectl apply -f $INGRESS/deploy/static/provider/kind/deploy.yaml
```

Verify the controller is ready (all options)

```
kubectl get pods -n ingress-nginx
kubectl wait --namespace ingress-nginx --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller --timeout=120s
kubectl get ingressclass          # should list: nginx
```

Step 6: The Ingress (ingress.yaml)

The Ingress is the routing rule: *"requests for host localhost and path / go to kuber-demo-service on port 80."*

ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: demo-ingress
spec:
  ingressClassName: nginx
  rules:
    - host: localhost
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: kuber-demo-service
                port:
                  number: 80
```

Every field explained

Field	What it means
`apiVersion: networking.k8s.io/v1`	The networking API group, where Ingress lives.
`kind: Ingress`	The type of object.
`metadata.name`	The name of this Ingress rule set.
`spec.ingressClassName: nginx`	Which Ingress Controller should handle this Ingress. Must match a class from `kubectl get ingressclass`.
`spec.rules`	A list of routing rules.
`host: localhost`	Only match requests whose **Host** header is `localhost`. Leave out `host` to match any host. For a custom name like `demo.local`, add it to your hosts file.
`http.paths`	The URL paths under this host.
`path: /`	The URL path to match.
`pathType: Prefix`	`Prefix` matches the path and everything under it (`/` matches all). `Exact` matches only that exact path. `ImplementationSpecific` leaves it to the controller.
`backend.service.name`	The Service to send traffic to: `kuber-demo-service`.
`backend.service.port.number`	The Service's **port** (80), not the container port (3000).

Step 7: Apply and test

1. Apply the files

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl apply -f ingress.yaml

# or all at once
kubectl apply -f .
```

2. Check that everything is up

```
kubectl get deployments           # READY should be 2/2
kubectl get pods                  # STATUS Running (2 Pods)
kubectl get svc kuber-demo-service
kubectl get endpoints kuber-demo-service   # must list 2 Pod IPs
kubectl get ingress               # ADDRESS shows localhost (may take a minute)
```

3. Open the app

```
curl http://localhost
# or open http://localhost in your browser
```

Quick check without the Ingress: this proves the app and Service work by themselves.

```
kubectl port-forward svc/kuber-demo-service 8080:80
# then open http://localhost:8080
```

Optional resources (the rest of the key terms)

These are not needed for the basic demo, but each appears in real projects. All examples are short and ready to apply.

Namespace

Groups resources so teams or environments do not collide. Without one, everything goes into `default`.

```
apiVersion: v1
kind: Namespace
metadata:
  name: demo
```

```
kubectl apply -f namespace.yaml
kubectl apply -f deployment.yaml -n demo
kubectl get pods -n demo
```

ConfigMap and Secret

ConfigMaps hold normal settings, Secrets hold sensitive values. Both can be injected as environment variables.

config.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "local"
  LOG_LEVEL: "info"
---
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
stringData:
  DB_PASSWORD: "change-me"
```

Use them in the Deployment

```
        # inside the container in deployment.yaml, next to "image:"
        envFrom:
        - configMapRef:
            name: app-config
        - secretRef:
            name: app-secret
```

> **Secrets are only base64-encoded, not encrypted.** Do not commit real secrets to Git. Anyone who can read the Secret object can decode it.

Persistent Volume (via a PersistentVolumeClaim)

A **PersistentVolumeClaim (PVC)** is a request for storage. Local clusters have a default StorageClass that creates the matching **PersistentVolume (PV)** automatically, so you rarely write a PV by hand.

pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 1Gi
```

Mount it in the Deployment

```
# in the Deployment: under the container, then under the Pod spec
        volumeMounts:
        - name: data
          mountPath: /data
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: app-data
```

Field	What it means
`accessModes: ReadWriteOnce`	Can be mounted read-write by one Node at a time. Others: `ReadOnlyMany`, `ReadWriteMany`.
`resources.requests.storage`	How much space you want (1 gibibyte).
`volumeMounts.mountPath`	The folder inside the container where the volume appears.

HPA (Horizontal Pod Autoscaler)

Adds Pods when CPU is high and removes them when it is low.

hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: kuber-demo-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: kuber-demo-deployment
  minReplicas: 2
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Field	What it means
`scaleTargetRef`	Which object to scale. Here: the Deployment from Step 3.
`minReplicas / maxReplicas`	The lowest and highest number of Pods the HPA may set.
`averageUtilization: 70`	Target average CPU use, measured against the Pod's CPU **request** (250m). Above about 175m average, it scales up.

HPA needs metrics-server. On minikube run `minikube addons enable metrics-server`. On Docker Desktop or kind, install it with the commands below. On local clusters you may also need to add the argument `--kubelet-insecure-tls` to the metrics-server Deployment. If `kubectl get hpa` shows `<unknown>`, metrics-server is missing or not ready.

```
MS=https://github.com/kubernetes-sigs/metrics-server/releases/latest/download
kubectl apply -f $MS/components.yaml
kubectl get deployment metrics-server -n kube-system
```

ServiceAccount and RBAC

A **ServiceAccount** is the identity a Pod runs as. **RBAC** (Role, RoleBinding) decides what that identity may do through the Kubernetes API. This example lets the app read Pods in its own namespace.

rbac.yaml

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-sa-pod-reader
subjects:
- kind: ServiceAccount
  name: app-sa
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Use it in the Deployment

```
# in the Deployment, under template.spec
      serviceAccountName: app-sa
```

Object / field	What it means
`ServiceAccount`	The identity of the Pod. Without one, Pods use the namespace's `default` account.
`Role`	A list of allowed actions inside **one namespace**. (`ClusterRole` works cluster-wide.)
`rules.resources / verbs`	Which objects (`pods`) and which actions (`get`, `list`, `create`, `delete`).
`RoleBinding`	Connects the Role to a subject (here, the ServiceAccount). A Role does nothing until it is bound.

Check the permission with: `kubectl auth can-i list pods --as=system:serviceaccount:default:app-sa`.

Everyday kubectl commands

Command	What it does
`kubectl get all`	Lists Pods, Services, Deployments and more in the current namespace
`kubectl describe pod <name>`	Full details and recent events. First stop when a Pod will not start
`kubectl logs <pod>`	Shows the container's output. Add `-f` to follow, `--previous` for the last crashed run
`kubectl exec -it <pod> -- sh`	Opens a shell inside the container
`kubectl scale deployment kuber-demo-deployment --replicas=3`	Changes the number of Pods by hand
`kubectl rollout restart deployment <name>`	Restarts all Pods gradually (picks up a new local image)
`kubectl rollout undo deployment <name>`	Rolls back to the previous version
`kubectl get events --sort-by=.lastTimestamp`	Recent cluster events, newest last
`kubectl delete -f .`	Deletes everything defined in the YAML files of the folder

Common problems

Problem	Likely cause and fix
`ErrImagePull` / `ImagePullBackOff`	The cluster cannot find the image. Load it with `minikube image load` or `kind load docker-image`, and keep `imagePullPolicy: IfNotPresent`. Check the image name for typos
`CrashLoopBackOff`	The app crashes on start. Run `kubectl logs <pod> --previous`. Often a wrong start command or a missing env var
Pod stuck in **Pending**	Not enough CPU or memory for the requests, or an unbound PVC. See the Events section of `kubectl describe pod`
`OOMKilled`	The container went over its memory limit. Raise `limits.memory` or fix the leak
Ingress returns **404** from nginx	Host or path does not match. Check `host: localhost` and that you are opening `http://localhost`, not an IP

Problem	Likely cause and fix
503 or **502** from nginx	The Service has no ready Pods. Run `kubectl get endpoints kuber-demo-service`. Empty means the selector does not match the Pod labels, or the Pods are not Ready
Connection refused on localhost:80	The controller is not exposed. kind: cluster was created without `extraPortMappings`. minikube: `minikube tunnel` is not running. Another program may already use port 80
Ingress has no ADDRESS	Controller not installed or not ready. Run `kubectl get pods -n ingress-nginx` and `kubectl get ingressclass`
Wrong port in the app	The Service `targetPort` (3000) must equal the port your Express app listens on
HPA shows <unknown>	metrics-server is not installed, or the container has no CPU `requests`

Cleanup

```
kubectl delete -f .                         # remove the app (Deployment, Service, Ingress)
helm uninstall ingress-nginx -n ingress-nginx   # only if you installed it with Helm

# remove the whole local cluster
kind delete cluster --name demo
minikube delete
# Docker Desktop: Settings > Kubernetes > uncheck Enable Kubernetes (or Reset cluster)
```

> Deleting the cluster removes everything in it, including PersistentVolumes created by the default StorageClass.